

Exercice-2

Pipeline : Extraction, nettoyage, qualité-validation et chargement de données en utilisant Python regex- Architecture Medallion

Consignes :

Les différentes étapes proposées dans cet énoncé constituent un guide méthodologique.

Important :

Vous n'êtes pas obligés de suivre exactement l'ordre ni l'ensemble des étapes décrites.

Vous êtes libres d'adopter votre propre démarche, à condition que celle-ci soit :

- Cohérente
- justifiée
- Complète

Il est fortement recommandé de :

- lire l'énoncé dans son intégralité avant de commencer
- structurer votre travail de manière logique

L'évaluation portera davantage sur:

- votre raisonnement
- votre capacité d'analyse
- la pertinence des traitements appliqués

Contexte

Vous êtes stagiaire mi-parcours dans une startup en croissance. Le responsable infrastructure vous remet un fichier de logs brut (access.log). Votre mission est de construire un pipeline qui alimente un lac de données structuré selon l'architecture Médaille, en vue d'analyses descriptives et de modélisation apprentissage machine (ML).

0.1 Architecture Medallion (lac de données à 3 couches)

Couche	Nom MongoDB	Contenu	Rôle
Bronze	Fichier access.log	Données brutes, non transformées	Source de vérité, archivage
Silver	Collection access_logs	Données nettoyées, enrichies, validées	Prêtes pour l'analyse
Gold	5 collections agrégées	Indicateurs, agrégats, alertes	Tableaux de bord, ML

Objectifs :

- Extraire et valider les données brutes des logs (Bronze)
- Nettoyer, enrichir et charger les données dans MongoDB (Silver)
- Calculer des indicateurs analytiques via MongoDB Aggregation Pipeline ou Pandas (Gold)
- Préparer un jeu de données exploitable pour :
 - Analyses descriptives : statistiques, visualisations, tendances de trafic, etc.
 - Machine Learning : détection d'anomalies, classification de trafic, prédictions de pannes, etc.

Format des logs — Extended Common Log Format (ECLF)

```
IP identd user [date] "MÉTHODE URL PROTOCOLE" status taille "referrer"
"user-agent"
```

Exemple de lignes :

```
192.168.1.10 - alice [10/Oct/2024:13:55:36 -0700] "GET /index.html
HTTP/1.1" 200 2326 "https://google.com" "Mozilla/5.0 (Windows NT 10.0)"

203.0.113.5 - - [10/Oct/2024:14:01:22 -0700] "POST /api/login HTTP/1.1"
401 512 "-" "curl/7.68.0"

999.999.999.999 - - [10/Oct/2024:14:05:00 -0700] "GET /hack HTTP/1.1" 404
128 "-" "python-requests/2.25"
```

1. Installation et Structure du Projet

1.1 Modules à installer

Exécutez les commandes suivantes dans l'Anaconda Prompt ou un terminal Python :

```
pip install geoip2 maxminddb-geolite2
```

Modules nécessaires:

Module	Usage	Commande d'installation
pymongo	Driver officiel MongoDB pour Python	pip install pymongo
geoip2	Lookup géographique des adresses IP	pip install geoip2
maxminddb-geolite2	Base GeoLite2 embarquée (sans compte)	pip install maxminddb-geolite2
pandas	Agrégations Gold (alternative à MongoDB)	pip install pandas
re	Expressions régulières	Inclus dans Python
datetime	Manipulation des dates	Inclus dans Python
os	Accès fichiers	Inclus dans Python

1.2 Imports des modules nécessaires :

```
import re                # Expressions régulières
import os                # Accès fichiers système
from datetime import datetime  # Conversion et extraction de dates
from pymongo import MongoClient  # Connexion et opérations MongoDB

# GeoIP : lookup pays depuis IP
try:
    from geolite2 import geolite2
    GEOIP_AVAILABLE = True
except ImportError:
    GEOIP_AVAILABLE = False      # Mode simulation activé

# Pandas : option pour les agrégations Gold
try:
    import pandas as pd
    PANDAS_AVAILABLE = True
except ImportError:
    PANDAS_AVAILABLE = False
```

1.3 Structure du projet

```
projet_TP_3/

pipeline.py      Pipeline (Bronze, Silver, Gold)
access.log       Couche Bronze : fichier brut
rapport.docx     Instructions d'exécution
```

Les collections MongoDB créées automatiquement :

```
weblog_db/

access_logs      Couche Silver
top_ips          Couche Gold
top_pages        Couche Gold
top_user_agents  Couche Gold
traffic_by_hour  Couche Gold
suspicious_ips   Couche Gold
```

2. Partie 1 : Description des données (Couche Bronze)

2.1 Ensemble de données :

Soit le fichier access.log constitue la couche Bronze du lac de données : données brutes, non filtrées, telles qu'elles sortent du serveur web

2.2 Description des champs du fichier access.log :

#	Champ	Type attendu	Exemple	Remarques / Cas particuliers
1	ip	str (IPv4)	192.168.1.10	Peut-être invalide (ex: 999.x.x.x) à filtrer
2	identd	str	-	Toujours '-' en pratique ignoré dans le pipeline
3	user	str ou -	alice	'-' = utilisateur anonyme sera converti en null
4	timestamp	str ECLF	10/Oct/2024:13:55:36-0700	À convertir en yyyy-MM-dd HH:mm:ss
5	method	str HTTP	GET	GET, POST, PUT, DELETE, HEAD, OPTIONS
6	url	str chemin	/index.html?q=1	Peut contenir des query params à normaliser
7	protocol	str	HTTP/1.1	HTTP/1.0, HTTP/1.1, HTTP/2.0
8	status_code	int 3 chiffres	200	Classes : 200,201(succès), 3xx(redirections), 4xx (erreurs client), 5xx (erreur serveur).
9	size	int ou -	2326	Taille réponse en octets. '-' = 0 octet
10	referrer	str URL ou -	https://google.com	'-' = accès direct sera converti en null
11	user_agent	str	Mozilla/5.0 ...	Navigateur, robot, scanner, outil CLI

3. Partie 2 : Extraction par Regex

3.1 Objectif

Analyser chaque ligne du fichier access.log avec une expression régulière pour en extraire les champs du format ECLF. C'est la phase extraction du pipeline.

3.2 Le pattern regex ECLF à construire

Construisez une regex compilée (re.compile) qui capture les groupes suivants :

Groupe	Champ	Pattern	Explication
1	ip	\S+	Tout caractère non-espace
-	identd	\S+	Ignoré (souvent tiret)
2	user	\S+	Identifiant ou tiret
3	raw_timestamp	\[(.+?)\]	Entre crochets, non-greedy
4	method	"(\S+)	Après le guillemet ouvrant
5	url	\S+	Chemin de la requête

Groupe	Champ	Pattern	Explication
6	protocol	\S+"	Avant le guillemet fermant
7	status_code	\d{3}	Exactement 3 chiffres
8	size	\d+ -	Entier ou tiret si inconnu
9	referrer	"(["]*)"	Entre guillemets
10	user_agent	"(["]*)"	Entre guillemets

3.3 Méthode à implémenter

Méthode	Entrée	Sortie	Comportement
parse_log (line)	str : une ligne brute du fichier log	dict avec les champs OU None	Applique LOG_PATTERN.match(). Retourne None si la ligne est vide ou ne correspond pas au pattern. Convertit status_code en int et size en int (0 si tiret).

Structure du dictionnaire retourné :

```
{ "ip": "192.168.1.10",
  "user": "alice",
  "raw_timestamp": "10/Oct/2024:13:55:36 -0700",
  "method": "GET",
  "url": "/index.html?q=1",
  "protocol": "HTTP/1.1",
  "status_code": 200,
  "size": 2326,
  "referrer": "https://google.com",
  "user_agent": "Mozilla/5.0 (Windows NT 10.0)" }
```

4. Partie 3 : Nettoyage et transformation

4.1 Objectif

Valider, nettoyer et enrichir chaque enregistrement parsé. C'est la phase de transformation qui produit des données prêtes pour la couche Silver.

4.2 Méthodes à implémenter

Méthode	Entrée	Sortie	Règle métier
is_valid_ipv4(ip)	str ip	bool	Regex stricte : chaque octet doit être entre 0 et 255. Rejeter 999.x.x.x, 256.x.x.x, etc.
convert_timestamp(raw_ts)	str '10/Oct/2024:13:55:36 - 0700'	str '2024-10-10 13:55:36' ou None	Utiliser datetime.strptime avec %d/%b/%Y:%H:%M:%S %z puis strftime('%Y-%m-%d %H:%M:%S'). Retourne None si malformé.
extract_hour(raw_ts)	str timestamp brut Apache	int 0-23 ou None	Extraire uniquement l'heure pour permettre l'agrégation temporelle en couche Gold.
get_status_category(status_code)	int ex: 404	str ex: 'Client Error'	Division entière : status_code // 100 clé dans le dict {2: Success, 3: Redirection, 4: Client Error, 5: Server Error}.
get_country(ip)	str ip valide	str nom du pays	1) Si IP privée (10.x, 192.168.x, 172.16-31.x, 127.x) Alors 'Private/Local'. 2) Sinon utiliser geolite2.reader().get(ip). 3) Si indisponible Alors simulation par premier octet.
normalize_url(url)	str '/search?q=python&p=2'	str '/search'	url.split('?')[0] supprime les paramètres de requête pour regrouper les pages.
clean_transform(record)	dict issu de parse_log	dict enrichi ou None	Orchestre toutes les transformations ci-dessus. Retourne None si IP invalide ou timestamp incorrect. Remplace '-' par None pour user et referrer.

4.3 Structure du document Silver (inséré dans MongoDB)

```
{ "ip": "203.0.113.5",
"user": null,
"timestamp": "2024-10-10 14:01:22",
"hour": 14,
"method": "POST",
"url": "/api/login",
"protocol": "HTTP/1.1",
"status_code": 401,
"status_category": "Client Error",
"size": 512,
"referrer": null,
"user_agent": "curl/7.68.0",
"country": "United States" }
```

5. Partie 4 : Chargement dans MongoDB Silver

5.1 Objectif

Charger les enregistrements nettoyés dans la collection access_logs (couche Silver de MongoDB).

5.2 Méthodes à implémenter

Méthode	Entrée	Sortie	Détails
get_db()	aucune	objet Database pymongo	Connexion à mongodb://localhost:27017, base weblog_db.
load_to_mongodb(records)	list[dict] enregistrements nettoyés	int nombre de documents insérés	collection.drop() puis insert_many(records). Utiliser insert_many Afficher le nombre de documents insérés.

5.3 Paramètres de connexion

Paramètre	Valeur
URI MongoDB	mongodb://localhost:27017
Base de données	weblog_db
Collection Silver	access_logs

6. Partie 5 : Agrégations Gold (Choix MongoDB ou Pandas)

6.1 Objectif

Calculer 5 indicateurs analytiques à partir de la couche Silver et les stocker dans des collections Gold.

- Les 10 IP les plus actives

- Les 10 pages les plus visitées(consultées)
- Les 5 User Agents les plus utilisés(fréquents).
- Volume de trafic par heure (0-23)
- Détection des IP suspectes (plus de 100 requêtes)

Vous avez le choix de l'outil d'agrégation

Approche	Outil	Avantage	Recommandée si
Option A	MongoDB Aggregation Pipeline	Traitement côté serveur, optimal pour gros volumes	Volume > 100 000 lignes
Option B	Pandas (Python)	Code plus lisible, idéal pour exploration et ML	Volume raisonnable, usage ML

6.2 Collections Gold à créer

Collection	Champs requis	Critère	Méthode MongoDB	Méthode Pandas
top_ips	ip, request_count	10 IP les plus actives	create_top_ips_mongo(db)	create_top_ips_pandas(df, db)
top_pages	url, visit_count	10 pages GET les plus visitées	create_top_pages_mongo(db)	create_top_pages_pandas(df, db)
top_user_agents	user_agent, count	5 UA les plus fréquents	create_top_ua_mongo(db)	create_top_ua_pandas(df, db)
traffic_by_hour	hour (0-23), request_count	Volume par heure	create_traffic_mongo(db)	create_traffic_pandas(df, db)
suspicious_ips	ip, request_count, flagged_at	IP avec > 100 requêtes	create_suspicious_mongo(db)	create_suspicious_pandas(df, db)

6.3 Signature des méthodes : Option A : MongoDB

Méthode	Entrée	Sortie	Opérateurs MongoDB utilisés
create_top_ips_mongo(db)	objet Database	None (écrit dans top_ips)	\$group, \$sum:1, \$sort, \$limit:10, \$project, \$out
create_top_pages_mongo(db)	objet Database	None (écrit dans top_pages)	\$match method:GET, \$group, \$sort, \$limit:10, \$out

Méthode	Entrée	Sortie	Opérateurs MongoDB utilisés
create_top_ua_mongo(db)	objet Database	None (écrit dans top_user_agents)	\$group, \$sort, \$limit:5, \$out
create_traffic_mongo(db)	objet Database	None (écrit dans traffic_by_hour)	\$group sur \$hour, \$sort _id:1, \$out
create_suspicious_mongo(db)	objet Database	None (écrit dans suspicious_ips)	\$group, \$match \$gt:100, \$project avec \$literal pour flagged_at, \$out

6.4 Signature des méthodes : Option B : Pandas

Méthode	Entrée	Sortie	Opération Pandas
load_silver_to_dataframe(db)	objet Database	pd.DataFrame	pd.DataFrame(list(db.access_logs.find())) — charge Silver en DataFrame
create_top_ips_pandas(df, db)	DataFrame, Database	None (écrit dans top_ips)	df.groupby('ip').size().nlargest(10).reset_index()
create_top_pages_pandas(df, db)	DataFrame, Database	None (écrit dans top_pages)	df[df.method=='GET'].groupby('url').size().nlargest(10)
create_top_ua_pandas(df, db)	DataFrame, Database	None (écrit dans top_user_agents)	df.groupby('user_agent').size().nlargest(5)
create_traffic_pandas(df, db)	DataFrame, Database	None (écrit dans traffic_by_hour)	df.groupby('hour').size().sort_index()
create_suspicious_pandas(df, db)	DataFrame, Database	None (écrit dans suspicious_ips)	Compter par IP, filtrer > 100, ajouter colonne flagged_at

7. Partie 6 : Pipeline complet : Appel des méthodes

7.1 Objectif

Appeler toutes les méthodes dans un pipeline run_pipeline() qui orchestre Bronze, Silver, Gold.

7.2 Méthode principale

Méthode	Entrée	Étapes internes
run_pipeline(log_file, use_pandas)	str chemin du fichier log, bool choix Pandas	1.Lecture du fichier ligne par ligne 2.parse_log_line() sur chaque ligne 3.clean_transform() sur chaque parsé 4.load_to_mongodb() pour charger Silver 5.Créer les 5 collections Gold (MongoDB ou Pandas)

Vérification dans mongosh

```
use weblog_db
show collections
db.access_logs.findOne()
db.top_ips.find().pretty()
db.suspicious_ips.find().pretty()
db.traffic_by_hour.find().sort({hour:1}).pretty()
```

Barème et Critères d'Évaluation

Partie	Critère évalué	Points
Partie 1 : Installer, importer les modules et charger acces.log	Génération correcte du fichier access.log avec toutes les variantes demandées	5
Partie 2 :Regex	Pattern LOG_PATTERN complet et fonctionnel sur les champs ECLF	20
Partie 2 : Regex	parse_log_line retourne None sur les lignes malformées	5
Partie3 : Validation IP	Regex IPv4 stricte (0-255 par octet, rejet de 999.x.x.x)	10
Partie 3 : Timestamp	Conversion correcte vers yyyy-MM-dd HH:mm:ss avec strptime/strftime	10
Partie 3 : Enrichissement	Catégorie HTTP correcte + pays via GeoIP ou simulation	10
Partie 3 :Nettoyage	Normalisation URL, remplacement des '-', extraction de l'heure	10
Partie 4 : MongoDB Silver	Connexion, drop + insert_many, collection bien structurée	10
Partie 5: Gold (5 collections)	Toutes les collections créées avec les bons champs (2 pts chacune)	10
Partie 5 : Choix Pandas ou Mongo	Les deux options fonctionnelles avec flag use_pandas	5
Partie 6 :Pipeline complet	run_pipeline() orchestre les différentes étapes	5